

# Chapter 15

## Automatic Scaling of Pods & Cluster Nodes

부하에 따라 Pod·노드를 자동으로 늘리고 줄이는 법

핵심 질문 — 누가 · 무엇을 보고 · 무엇을 조절하는가?

# 오늘의 세 축



## HPA

Horizontal Pod Autoscaler

Pod **개수**를 조절  
(replica ↑↓)



## VPA

Vertical Pod Autoscaler · 애드온

Pod의 **requests/limits**  
(크기) 조절



## Cluster Autoscaler

클라우드 프로바이더 연동

**노드** 개수를 조절

**HPA = 가로(개수), VPA = 세로(자원), CA = 노드.** 이 세 단어면 구조가 잡힌다.

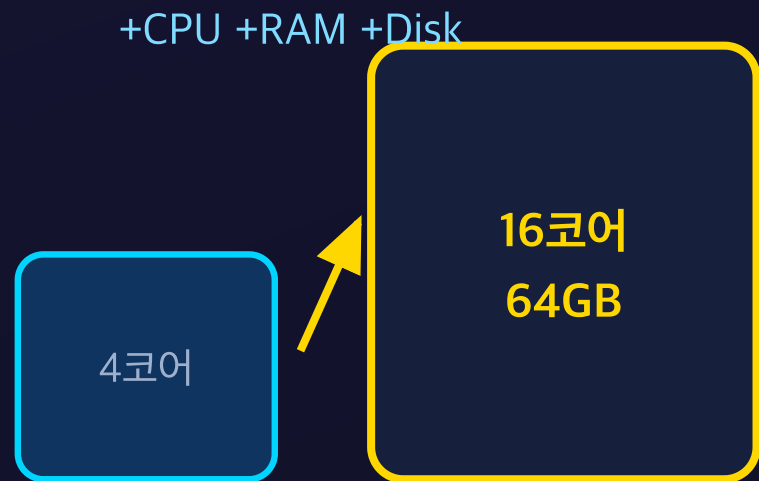
HPA ↔ CA는 **협력**(Pod↑→Pending→노드↑), HPA ↔ VPA(동일 자원)는 **충돌**.

# 01

## 수평 vs 수직 확장

쿠버네티스 이전부터 모든 시스템에 공통인 두 방향

## 수직 확장 — 기계를 더 키운다



### ✅ 장점

- 단순함 — 코드 그대로
- 분산 복잡성 없음
- **stateful·RDB**에 적합

### ⚠️ 단점

- 하드웨어 상한
- SPOF
- 보통 재부팅/다운타임
- 고사양일수록 **비용 폭증**

예: DB를 4코어/16GB → 16코어/64GB로 교체 (RDS 인스턴스 클래스 업그레이드)

## 수평 확장 — 기계를 더 늘린다



### ✅ 장점

- 거의 선형 확장
- HA — 한 대 죽어도 OK
- 범용 장비 (클라우드 친화)

### ⚠️ 단점

- 앱이 무상태 전제
- LB·데이터 일관성
- 네트워크 → 복잡성↑

예: 웹 서버 1대 → 10대 + LB. DB는 read replica / 샤딩.

# 핵심 트레이드오프

|       | ↕ 수직 (Up)        | ↔ 수평 (Out)          |
|-------|------------------|---------------------|
| 상한    | 하드웨어 한계          | 사실상 무제한             |
| 가용성   | SPOF             | HA(분산)              |
| 복잡성   | 낮음(코드 그대로)       | 높음(분산 설계)           |
| 다운타임  | 보통 필요            | 보통 무중단              |
| 적합    | RDB·stateful·레거시 | stateless 웹/API·MSA |
| 비용 곡선 | 고사양일수록 급증        | 범용 장비로 완만           |

클라우드 네이티브는 수평 선호. 단 DB 등 stateful은 여전히 수직이 1차 → 그래서 k8s는 둘 다 제공.

# 02

## Pod 단위 스케일링

Pod 자체를 조절하는 두 방향  
수평(HPA, 개수) · 수직(VPA, 크기)

# HPA 컨트롤러는 어디에 있나

kube-controller-manager

HPA 루프

15초마다

- ① 메트릭 조회
- ② replica 계산
- ③ scale 수정

- HPA는 **별도 데몬/Pod이 아니다** — 컨트롤 루프 하나.
- 대상의 **scale 서브리소스**(Deployment/RS/STS)를 수정.
- 주기: `--horizontal-pod-autoscaler-sync-period` (기본 15s)

API: `autoscaling/v1` · `v2` = apiserver built-in



# Pod 메트릭은 어떻게 수집되나

자원 사용량의 원천은 노드의 커널 cgroup. 위로 올라오는 경로:



- **cAdvisor**: 컨테이너 자원 사용량 수집·노출 데몬, **kubelet에 내장**(별도 설치 X).
- 리눅스는 **cgroup** 기반, 그 외엔 런타임의 CRI Container Metrics.

# kubelet 구성요소 — cAdvisor는 일부일 뿐

## 메트릭 관련 (오늘 주제)

- **cAdvisor** — 컨테이너별 CPU/메모리 수집
- Summary API — `/stats/summary` 요약
- `/metrics/resource` — metrics-server가 긁는 곳

## 그 외 kubelet이 하는 일

- **SyncLoop** — 원하는↔실제 Pod 상태 일치
- **PLEG** — 컨테이너 생성·종료 이벤트
- **CRI 클라이언트** — containerd/CRI-O에 기동 명령
- Volume/Image · Probe/Status · Eviction · Node Lease

**kubelet = 노드 위 컨테이너의 생사·상태·자원·볼륨 책임 에이전트.** cAdvisor는 그 안의 자원 측정 모듈일 뿐 — "왜 메트릭 엔드포인트가 kubelet에?"의 답.

# Heapster의 죽음 → 지금의 파이프라인

## ❌ 예전 (책, Heapster)

```
kubelet/cAdvisor  
  → Heapster (집계, InfluxDB)  
  → HPA가 Heapster 직접 호출
```

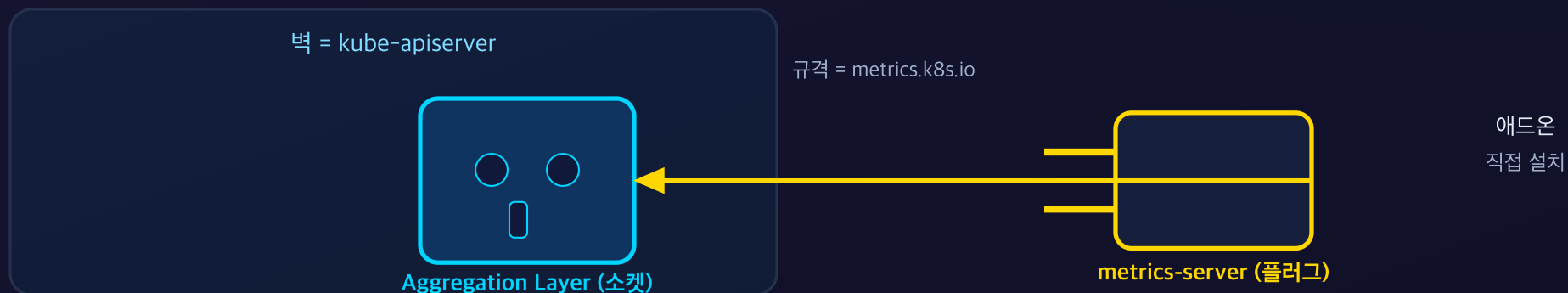
HPA가 특정 구현에 직접 의존 → 결합도↑·무거움.

2019 retired.

## ✅ 지금 (metrics-server)

```
kubelet(/metrics/resource)  
  ↑ metrics-server 15s scrape  
  (최근값만 메모리, 저장 X)  
  ↓ APIService 등록  
Aggregation Layer(apiserver 내장)  
  ↑ HPA / kubectl top 조회
```

# 인터페이스 · 소켓 · 구현체 — 콘센트 비유



- Aggregation Layer  apiserver 내장 / metrics-server  애드온 (안 깔면 `top` ·HPA 모두 `<unknown>` )
- **HPA가 core인 이유**: HPA는 **인터페이스**에 의존하지 구현(metrics-server)에 의존하지 않는다.

# HPA 계산식

$$\text{desiredReplicas} = \text{ceil}(\text{currentReplicas} \times (\text{currentMetric} / \text{desiredMetric}))$$

|         |   |           |   |         |
|---------|---|-----------|---|---------|
| 3       | × | 90%50%    | = | 6       |
| replica |   | 현재/목표 CPU |   | desired |

- CPU 사용률은 노드가 아니라 컨테이너 **requests 대비 %** → **requests 없으면 계산 불가** (ch14 직결).
- **tolerance 10%**: 비율 0.9~1.1이면 변동 없음. Not-Ready·missing metric은 보수 보정 → "공식대로면 6인데 5"도 정상.

# 메트릭 종류 3가지



## Resource

metrics.k8s.io (metrics-server)  
CPU, 메모리



## Pods / Object

custom.metrics.k8s.io (Prom Adapter)  
초당 요청수, 큐 길이



## External

external.metrics.k8s.io (KEDA)  
SQS·PubSub 메시지 수

여러 메트릭 지정 시 **각각 계산해 가장 큰 replica 채택**(안전한 쪽). 비즈니스 신호(QPS·큐·p99)로 스케일하면 CPU보다 정확.

# 스케일링 안정화 — flapping 방지

```
spec:
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300 # 5분 보수
      policies: [{type: Percent, value: 50,
                  periodSeconds: 60}]
    scaleUp:
      stabilizationWindowSeconds: 0 # 즉시
      policies: [{type: Pods, value: 4,
                  periodSeconds: 60}]
```

| 키                          | 역할                                                                                       |
|----------------------------|------------------------------------------------------------------------------------------|
| behavior                   | scaleUp · scaleDown 을 <b>방향별로 따로</b> 튜닝 (autoscaling/v2)                                 |
| stabilizationWindowSeconds | 이 시간 동안 추천값을 모아 <b>↓는 최댓값·↑는 최솟값</b> 채택 → 순간 변동 무시.<br>↓ <b>300</b> (5분)·↑ <b>0</b> (즉시) |
| policies                   | 변경 <b>속도 상한</b> 규칙 목록                                                                    |
| type                       | <b>Pods</b> =절대 개수 / <b>Percent</b> =현재 replica 대비 %                                     |
| value                      | 상한 크기 ( <b>Percent 50</b> =한 번에 절반까지)                                                    |
| periodSeconds              | 그 상한을 " <b>~초당</b> "으로 적용하는 시간창 (여기선 60초)                                                |

**예시 해석** — 축소: 5분 보수적으로 보다가 줄여도 **60초당 현재의 50%까지만**. 확대: 즉시 반응하되 **60초당 최대 4개까지만**. 정책이 여러 개면 **selectPolicy** (기본 **Max** =가장 큰 변화 허용)로 택1, **Disabled** 면 그 방향을 아예 잠금.

# VPA

## Vertical Pod Autoscaler

HPA가 개수를 늘린다면,  
VPA는 Pod 하나의 **requests/limits**를 실사용량에 맞춰 자동 조정



# 코어 빌트인 아님 + 3 컴포넌트

VPA는 코어 내장이 아니다. `kubernetes/autoscaler` 레포의 별도 애드온(직접 설치). API 그룹 `autoscaling.k8s.io`.



## Recommender

사용량 분석 → 적정 requests 추천값 계산



## Updater

추천값과 동떨어진 기존 Pod 축출(재생성 유도)



## Admission Controller

새 Pod에 추천값 주입(mutating webhook)

# updateMode 4가지

| 모드       | 동작                | 용도                      |
|----------|-------------------|-------------------------|
| Off      | 추천만, 변경 X         | 가장 안전 / HPA와 병행         |
| Initial  | 생성 시점에만 주입        | 재시작 부담 최소               |
| Recreate | Pod 죽이고 새 값으로 재생성 | 무중단 아님                  |
| Auto     | 현재 Recreate와 동일   | (향후 In-place resize 결합) |

- 가장 큰 함정

# HPA와 VPA의 충돌

같은 CPU 신호 →

HPA: replica ↑

VPA: requests ↑

반대로 반응 → 진동

같은 자원(CPU/메모리)에 HPA·VPA 동시 사용 금지.

## ✓ 허용 조합

- HPA(CPU) + VPA **Off** (추천만)
- HPA(커스텀 QPS) + VPA(CPU·메모리)

# 무중단 수직 스케일링의 미래

- 전통적 VPA는 requests 변경 = **Pod 재시작**.
- 코어 **In-place Pod Resize**( `resizePolicy` , 1.27 alpha → 1.33 beta)와 결합 시 **재시작 없이** 조정.

코어에 들어온 건 VPA가 아니라,  
자원을 재시작 없이 바꾸는 In-place Resize.

# 03

## Node 단위 스케일링

Pod을 늘렸는데(HPA) **들어갈 노드가 없으면?**

노드 자체를 늘리는 레이어

# Cluster Autoscaler (CA)

## ↑ Scale up

Pod이 자원 부족으로 **Pending**(스케줄 실패) → 노드 그룹에 노드 추가.

## ↓ Scale down

활용률 낮고(기본 50%↓) Pod 재배치 가능 → drain 후 노드 제거.

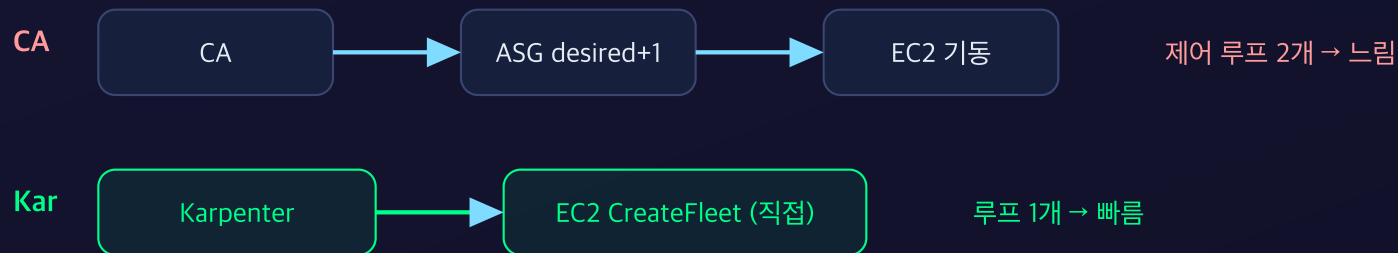
HPA가 replica↑ → Pod이 Pending(노드 부족) → CA가 노드 추가 → Pod 스케줄됨

**축소를 막는 요소**(비용 누수 단골): 빡빡한 PodDisruptionBudget · local storage Pod · `safe-to-evict:false` · 시스템 Pod.

# Karpenter vs Cluster Autoscaler

|    | Cluster Autoscaler            | Karpenter                                  |
|----|-------------------------------|--------------------------------------------|
| 단위 | 미리 정의된 <b>노드 그룹/ASG</b> 내 +/- | 그룹 없이 <b>Pending Pod</b> 에 딱 맞는 인스턴스 즉시 선택 |
| 속도 | 느림 (ASG 경유)                   | <b>빠름</b> (직접 프로비저닝)                       |
| 비용 | 그룹 단위 한계                      | <b>spot/타입 혼합 최적</b>                       |
| 범위 | 모든 클라우드/온프레임                  | 주로 AWS (확산 중)                              |

## 왜 Karpenter가 더 빠르나 — 근본 2가지



### ① 프로비저닝 간접성

CA는 ASG desired만 +1 → 실제 기동은 ASG에 **위임**(루프 2개). Karpenter는 EC2 API **직접** 호출.

### ② 노드그룹 = 고정 템플릿

CA는 그룹 중 **고르기**만 → 못 맞으면 재시도. Karpenter는 requests로 **최적 타입 즉시 계산**.

CA는 **이식성**(모든 클라우드) 위해 노드그룹 추상화를 거치느라 2단 루프+템플릿을 떠안음. **속도(이득) = 이식성 상실(비용)**, 같은 원인의 양면.



# Karpenter 도입하기 (EKS)

## ① 컨트롤러 설치 (Helm)

```
# IAM 역할·노드역할·태그는 eksctl/CFN로 선행
helm upgrade --install karpenter \
  oci://public.ecr.aws/karpenter/karpenter \
  --version 1.0.6 --n kube-system \
  --set settings.clusterName=$CLUSTER \
  --set settings.interruptionQueue=$CLUSTER
```

컨트롤러 자신은 관리형 노드그룹/Fargate에 1~2개만.

## ② 무엇을 어떻게 띄울지 선언

```
apiVersion: karpenter.sh/v1
kind: NodePool                                # 무엇을: 제약·한도
metadata: {name: default}
spec:
  template:
    spec:
      requirements:
        - {key: karpenter.sh/capacity-type, operator: In,
          values: ["spot", "on-demand"]}
      nodeClassRef: {group: karpenter.k8s.aws,
        kind: EC2NodeClass, name: default}
      limits: {cpu: "1000"}                    # 폭주 상한
      disruption: {consolidationPolicy: WhenEmptyOrUnderutilized}
---
apiVersion: karpenter.k8s.aws/v1
kind: EC2NodeClass                            # 어떻게: AMI·서브넷·역할
metadata: {name: default}
spec:
  role: KarpenterNodeRole-$CLUSTER
  amiSelectorTerms: [{alias: al2023@latest}]
  subnetSelectorTerms: [{tags: {karpenter.sh/discovery: $CLUSTER}}]
  securityGroupSelectorTerms: [{tags: {karpenter.sh/discovery: $CLUSTER}}]
```

**NodePool**=어떤 노드를 허용할지(타입·spot·한도), **EC2NodeClass**=그 노드를 AWS에서 어떻게 만들지(AMI·서브넷·IAM). **노드그룹을 미리 안 만든다** — 제약만 주면 Karpenter가 즉석에서 고름.

# Pod 늘리고 줄이기 → 노드가 따라온다

## ① 조작 (이게 트리거)

```
# 데모 워크로드 (pause, requests=1코어)
kubectl create deployment inflate \
  --image=registry.k8s.io/pause:3.9
kubectl set resources deploy/inflate --requests=cpu=1

# ▲ 늘리기 → Pending → Karpenter가 노드 생성
kubectl scale deploy/inflate --replicas=20

# ▼ 줄이기 → 빈/저활용 노드 자동 정리
kubectl scale deploy/inflate --replicas=0
```

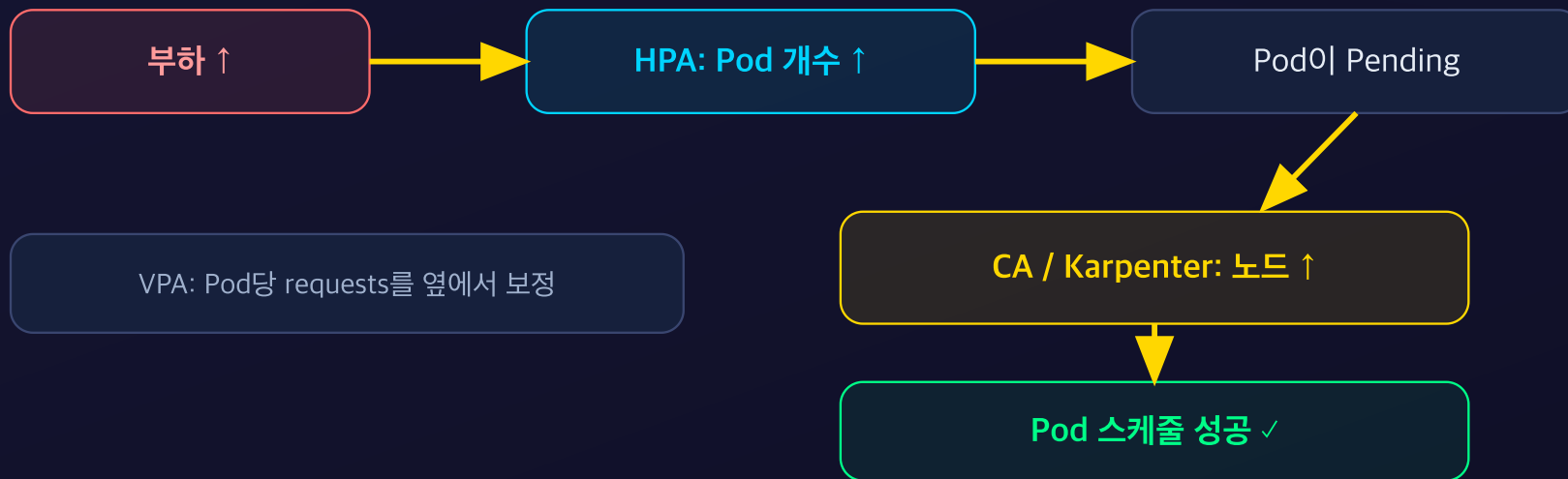
## ② 관찰

```
kubectl get pods -w      # Pending→Running
kubectl get nodeclaims    # Karpenter 노드요청
kubectl get nodes -w      # 노드 증감
eks-node-viewer          # 비용·빈패킹 실시간
kubectl logs -f -n kube-system \
  -l app.kubernetes.io/name=karpenter
```

**왜 requests가 필수?** 스케줄러가 자리를 requests로 계산 → **Pending** 판정이 곧 **Karpenter 입력**. requests 없으면 못 늘림 (슬라이드 13 연결).

**kubectl scale** 로 **replica**를 바꾸는 게 트리거. 실제 운영에선 이 replica를 **HPA가 자동으로** 올리고(부하↑), Karpenter가 노드를 받쳐줌 — HPA↔Karpenter 협력(다음 장 "세 층위 흐름").

## 세 층위의 협력 흐름



HPA ↔ CA는 협력(Pod↑→Pending→노드↑). HPA ↔ VPA(동일 자원)는 충돌.

# 책 vs 현실

| 책(2017, 1판)            | 현재(2024+)                                             |
|------------------------|-------------------------------------------------------|
| 메트릭 = <b>Heapster</b>  | <b>폐기(2019)</b> , metrics-server + Prometheus Adapter |
| HPA <b>v1</b> (CPU만)   | <b>v2</b> — 메모리·커스텀·External + <b>behavior</b>        |
| 앱 <b>idling</b> "계획 중" | <b>코어에 끝내 안 들어옴</b> . 진짜 idling은 Knative/KEDA         |
| VPA "계획 중"             | <b>애드온으로 실현</b> (코어 X), In-place Resize와 결합 중         |
| scale-to-zero 불가       | <b>KEDA</b> (이벤트 기반 0↔N, 60+ scaler)                  |
| Cluster Autoscaler만    | <b>Karpenter</b> 대안 부상                                |

# KEDA — 이벤트 기반 + scale-to-zero

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
spec:
  scaleTargetRef: { name: consumer }
  minReplicaCount: 0    # ← 0까지
  maxReplicaCount: 50
  triggers:
  - type: kafka
    metadata:
      topic: orders
      lagThreshold: "100"
```

기본 HPA는 `minReplicas:1` 이하로 못 내려간다. KEDA는 이벤트 없으면 **0으로 축소**, 오면 깨워 이후 HPA에 위임.

Kafka lag

SQS depth

RabbitMQ

cron

60+ scaler

# Key Takeaways

- **HPA=가로(개수), VPA=세로(자원), CA=노드** — 세 단어로 구조가 잡힌다.
- HPA 공식 `ceil(replica × 현재/목표)`, CPU는 **requests 대비** → requests 필수(ch14 직결).
- **HPA ↔ CA 협력, HPA ↔ VPA(동일 자원) 충돌** — VPA는 추천(Off) 모드로 병행.
- metrics-server는 **애드온** — 안 깔면 `top` ·HPA 모두 `<unknown>`.
- flapping은 `behavior` 로, scale-to-zero는 **KEDA**로, 노드는 **CA/Karpenter**로.

# Q & A

HPA · VPA · Cluster Autoscaler · Karpenter · KEDA

감사합니다 🙌